# Book Metadata and Recommendation API Documentation

**Course:** Web Services and Web Data (XJCO3011)
**Date:** April 2026

---

## Table of Contents


1. Overview
2. Getting Started
3. Authentication
4. API Endpoints
    - Authors
    - Genres
    - Books
    - Reviews
    - Reading Lists
    - Search
    - Recommendations
    - Statistics
5. Data Models
6. Error Handling
7. Examples


---

## Overview

### Description

The Book Metadata and Recommendation API is a comprehensive RESTful web service designed for managing book metadata, user reviews, and providing intelligent book recommendations. Built with Django REST Framework, this API offers full CRUD operations and advanced features including search, filtering, and content-based recommendations.

### Key Features

- **Complete CRUD Operations** - Create, Read, Update, Delete for all resources
- **Advanced Search** - Full-text search across books, authors, and genres
- **Smart Recommendations** - Content-based recommendation algorithm
- **Review System** - User ratings and comments
- **Statistics** - Real-time analytics and insights
- **Reading Lists** - Personal book collection management
- **Authentication** - Secure user authentication for protected endpoints

### Technology Stack

- **Framework:** Django 6.0.3
- **API Framework:** Django REST Framework 3.17.1
- **Database:** SQLite
- **Documentation:** drf-yasg (Swagger/OpenAPI)
- **Additional Libraries:** django-filter

### Base URL

```
http://127.0.0.1:8000/api/
```

---

## Getting Started

### Installation

1. **Clone the repository:**

```
git clone <repository-url>
cd bookapi
```

2. **Install dependencies:**

```
pip install django djangorestframework django-filter drf-yasg
```

3. **Run migrations:**

```
python manage.py migrate
```

4. **Load sample data:**

```
python manage.py load_sample_data
```

5. **Start the server:**

```
python manage.py runserver
```

### Quick Start

Access the interactive API documentation:

- **Swagger UI:** http://127.0.0.1:8000/swagger/
- **ReDoc:** http://127.0.0.1:8000/redoc/

---

# Authentication

## Overview

The API uses Django's built-in authentication system. Some endpoints require authentication while others are publicly accessible.

## Authentication Methods

- **Session Authentication** - For browser-based access
- **Basic Authentication** - For API clients

## Protected Endpoints

The following endpoints require authentication:

- `POST /api/reviews/` - Create a review
- `PUT/PATCH /api/reviews/{id}/` - Update own review
- `DELETE /api/reviews/{id}/` - Delete own review
- `GET/POST /api/reading-list/` - Manage reading list

## Public Endpoints

All other endpoints are publicly accessible for read operations.

---

# API Endpoints

## Authors

### List All Authors

**Endpoint:** `GET /api/authors/`

**Description:** Retrieve a paginated list of all authors.

**Query Parameters:**

- `search` (string, optional) - Search by author name
- `ordering` (string, optional) - Order by field (e.g., `name`, `-created_at`)
- `page` (integer, optional) - Page number for pagination

**Response:** `200 OK`

```
{
  "count": 7,
  "next": null,
```

```
  "previous": null,
  "results": [
    {
      "id": 1,
      "name": "J.K. Rowling",
      "biography": "",
      "birth_date": "1965-07-31",
      "nationality": "British",
      "books_count": 1,
      "created_at": "2026-04-13T10:37:31.257006Z",
      "updated_at": "2026-04-13T10:37:31.257006Z"
    }
  ]
}
```

---

#### Get Author Details

**Endpoint:** `GET /api/authors/{id}/`

**Description:** Retrieve detailed information about a specific author.

**Path Parameters:**

- `id` (integer, required) - Author ID

**Response:** `200 OK`

```
{
  "id": 1,
  "name": "J.K. Rowling",
  "biography": "",
  "birth_date": "1965-07-31",
  "nationality": "British",
  "books_count": 1,
  "created_at": "2026-04-13T10:37:31.257006Z",
  "updated_at": "2026-04-13T10:37:31.257006Z"
}
```

---

#### Create Author

**Endpoint:** `POST /api/authors/`

**Description:** Create a new author.

**Request Body:**

```
{
  "name": "New Author",
  "biography": "Author biography",
  "birth_date": "1980-01-01",
  "nationality": "American"
}
```

**Response:** `201 Created`

```
{
  "id": 8,
  "name": "New Author",
  "biography": "Author biography",
  "birth_date": "1980-01-01",
  "nationality": "American",
  "books_count": 0,
  "created_at": "2026-04-19T15:30:00.000000Z",
  "updated_at": "2026-04-19T15:30:00.000000Z"
}
```

---

#### Update Author

**Endpoint:** `PUT /api/authors/{id}/`

**Description:** Update an existing author (full update).

**Path Parameters:**

- `id` (integer, required) - Author ID

**Request Body:**

```
{
  "name": "Updated Author Name",
  "biography": "Updated biography",
  "birth_date": "1980-01-01",
  "nationality": "British"
}
```

**Response:** `200 OK`

---

#### Partial Update Author

**Endpoint:** `PATCH /api/authors/{id}/`

**Description:** Partially update an author.

**Request Body:**

```
{
  "biography": "Updated biography only"
}
```

**Response:** `200 OK`

---

**Delete Author**

**Endpoint:** `DELETE /api/authors/{id}/`

**Description:** Delete an author.

**Response:** `204 No Content`

---

## Genres

**List All Genres**

**Endpoint:** `GET /api/genres/`

**Description:** Retrieve a list of all book genres.

**Query Parameters:**

- `search` (string, optional) - Search by genre name
- `ordering` (string, optional) - Order results

**Response:** `200 OK`

```
{
  "count": 7,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 1,
      "name": "Fantasy",
      "description": "Fantasy literature",
      "books_count": 1,
      "created_at": "2026-04-13T10:37:31.304068Z"
    }
  ]
}
```

---

#### Get Genre Details

**Endpoint:** `GET /api/genres/{id}/`

**Response:** `200 OK`

---

#### Create Genre

**Endpoint:** `POST /api/genres/`

**Request Body:**

```
{
  "name": "Science Fiction",
  "description": "Science fiction and speculative fiction"
}
```

**Response:** `201 Created`

---

#### Update/Delete Genre

Similar to Authors endpoints.

---

### Books

#### List All Books

**Endpoint:** `GET /api/books/`

**Description:** Retrieve a paginated list of all books with filtering and search.

**Query Parameters:**

- `search` (string, optional) - Search in title, description, author name, publisher
- `genres` (integer, optional) - Filter by genre ID
- `authors` (integer, optional) - Filter by author ID
- `language` (string, optional) - Filter by language
- `ordering` (string, optional) - Order by: `title`, `publication_date`, `average_rating`, `-created_at`
- `page` (integer, optional) - Page number

**Response:** `200 OK`

```
{
  "count": 7,
  "next": null,
```

```
  "previous": null,
  "results": [
    {
      "id": 1,
      "title": "Harry Potter and the Philosopher's Stone",
      "authors": [
        {
          "id": 1,
          "name": "J.K. Rowling",
          "nationality": "British"
        }
      ],
      "genres": [
        {
          "id": 1,
          "name": "Fantasy"
        }
      ],
      "description": "The first book in the Harry Potter series.",
      "isbn": "9780747532699",
      "publisher": "Bloomsbury",
      "publication_date": "1997-06-26",
      "page_count": 223,
      "language": "English",
      "average_rating": "3.00",
      "ratings_count": 1,
      "cover_image_url": "",
      "created_at": "2026-04-13T10:37:31.345123Z"
    }
  ]
}
```

---

#### Get Book Details

**Endpoint:** `GET /api/books/{id}/`

**Description:** Retrieve detailed information about a specific book, including all reviews.

**Response:** `200 OK`

```
{
  "id": 1,
  "title": "Harry Potter and the Philosopher's Stone",
  "authors": [...],
  "genres": [...],
  "description": "The first book in the Harry Potter series.",
  "isbn": "9780747532699",
  "publisher": "Bloomsbury",
  "publication_date": "1997-06-26",
  "page_count": 223,
```

```
    "language": "English",
    "average_rating": "3.00",
    "ratings_count": 1,
    "reviews": [
      {
        "id": 1,
        "user": {
          "id": 4,
          "username": "testuser3"
        },
        "rating": 3,
        "comment": "One of my favorite books of all time.",
        "created_at": "2026-04-13T10:37:36.406191Z"
      }
    ],
    "cover_image_url": "",
    "created_at": "2026-04-13T10:37:31.345123Z",
    "updated_at": "2026-04-13T10:37:31.345123Z"
  }
```

---

**Create Book**

**Endpoint:** `POST /api/books/`

**Request Body:**

```
  {
    "title": "New Book Title",
    "description": "Book description",
    "isbn": "9781234567890",
    "publisher": "Publisher Name",
    "publication_date": "2026-01-01",
    "page_count": 300,
    "language": "English",
    "author_ids": [1, 2],
    "genre_ids": [1, 3]
  }
```

**Response:** `201 Created`

---

**Update/Delete Book**

Similar patterns as Authors.

---

## Reviews

**List All Reviews**

**Endpoint:** `GET /api/reviews/`

**Description:** Retrieve all book reviews.

**Query Parameters:**

- `book` (integer, optional) - Filter by book ID
- `user` (integer, optional) - Filter by user ID
- `ordering` (string, optional) - Order by field

**Response:** `200 OK`

```
{
  "count": 12,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 1,
      "book": 1,
      "book_title": "Harry Potter and the Philosopher's Stone",
      "user": {
        "id": 4,
        "username": "testuser3",
        "email": "testuser3@example.com"
      },
      "rating": 3,
      "comment": "One of my favorite books of all time.",
      "created_at": "2026-04-13T10:37:36.406191Z",
      "updated_at": "2026-04-13T10:37:36.406191Z"
    }
  ]
}
```

---

**Create Review**

**Endpoint:** `POST /api/reviews/`

**Authentication:** Required

**Request Body:**

```
{
  "book": 1,
  "rating": 5,
  "comment": "Excellent book! Highly recommended."
}
```

**Response:** `201 Created`

**Validation:**

- `rating` must be between 1 and 5
- User can only have one review per book

---

**Update Review**

**Endpoint:** `PUT /api/reviews/{id}/`

**Authentication:** Required (must be review owner)

---

**Delete Review**

**Endpoint:** `DELETE /api/reviews/{id}/`

**Authentication:** Required (must be review owner)

**Response:** `204 No Content`

---

## Reading Lists

**Get User's Reading List**

**Endpoint:** `GET /api/reading-list/`

**Authentication:** Required

**Description:** Retrieve the authenticated user's reading list.

**Response:** `200 OK`

```
{
  "count": 3,
  "results": [
    {
      "id": 1,
      "book": {
        "id": 1,
        "title": "Harry Potter and the Philosopher's Stone",
        "authors": [...]
      },
      "status": "reading",
      "added_at": "2026-04-13T10:37:40.123456Z"
    }
  ]
}
```

---

#### Add Book to Reading List

**Endpoint:** `POST /api/reading-list/`

**Authentication:** Required

**Request Body:**

```
{
  "book": 1,
  "status": "want_to_read"
}
```

**Status Options:**

- `want_to_read`
- `reading`
- `completed`

**Response:** `201 Created`

---

#### Update Reading List Item

**Endpoint:** `PATCH /api/reading-list/{id}/`

**Request Body:**

```
{
  "status": "completed"
}
```

---

#### Remove from Reading List

**Endpoint:** `DELETE /api/reading-list/{id}/`

**Response:** `204 No Content`

---

### Search

#### Search Books

**Endpoint:** `GET /api/search/`

**Description:** Full-text search across books, authors, genres, and publishers.

**Query Parameters:**

- `q` (string, required) - Search query

**Response:** `200 OK`

```
{
  "query": "harry",
  "count": 1,
  "results": [
    {
      "id": 1,
      "title": "Harry Potter and the Philosopher's Stone",
      "authors": [...],
      "genres": [...],
      "average_rating": "3.00",
      "publication_date": "1997-06-26"
    }
  ]
}
```

**Search Fields:**

- Book title
- Book description
- Author names
- Genre names
- Publisher name

---

## Recommendations

### Get Book Recommendations

**Endpoint:** `GET /api/books/{id}/recommendations/`

**Description:** Get personalized book recommendations based on a specific book.

**Path Parameters:**

- `id` (integer, required) - Book ID

**Response:** `200 OK`

```
{
  "book": {
    "id": 1,
    "title": "Harry Potter and the Philosopher's Stone",
    "authors": [...],
    "genres": [...]
```

```
    },
    "recommendations": [
      {
        "id": 2,
        "title": "Similar Book",
        "authors": [...],
        "genres": [...],
        "similarity_score": 0.85,
        "average_rating": "4.50"
      }
    ]
  }
```

**Algorithm:**

- Content-based filtering
- Calculates similarity based on shared genres and authors
- Returns top 5 most similar books
- Excludes the original book

---

## Statistics

### Get Popular Books

**Endpoint:** `GET /api/popular/`

**Description:** Retrieve the most popular books based on ratings.

**Query Parameters:**

- `limit` (integer, optional) - Number of results (default: 10)

**Response:** `200 OK`

```
{
  "popular_books": [
    {
      "id": 3,
      "title": "Pride and Prejudice",
      "authors": [...],
      "average_rating": "4.00",
      "ratings_count": 2
    }
  ]
}
```

---

### Get System Statistics

**Endpoint:** `GET /api/statistics/`

**Description:** Retrieve comprehensive system statistics.

**Response:** `200 OK`

```
{
  "total_books": 7,
  "total_authors": 7,
  "total_genres": 7,
  "total_reviews": 12,
  "average_rating": 3.58,
  "books_by_genre": [
    {
      "genre": "Classic",
      "count": 5
    }
  ],
  "top_rated_books": [
    {
      "id": 3,
      "title": "Pride and Prejudice",
      "average_rating": "4.00"
    }
  ],
  "most_reviewed_books": [
    {
      "id": 2,
      "title": "1984",
      "review_count": 2
    }
  ]
}
```

---

# Data Models

## Author

```
{
  "id": "integer (auto-generated)",
  "name": "string (required, max 200 chars)",
  "biography": "text (optional)",
  "birth_date": "date (optional, format: YYYY-MM-DD)",
  "nationality": "string (optional, max 100 chars)",
  "books_count": "integer (computed)",
  "created_at": "datetime (auto-generated)",
  "updated_at": "datetime (auto-updated)"
}
```

---

### Genre

```
{
  "id": "integer (auto-generated)",
  "name": "string (required, unique, max 100 chars)",
  "description": "text (optional)",
  "books_count": "integer (computed)",
  "created_at": "datetime (auto-generated)"
}
```

---

### Book

```
{
  "id": "integer (auto-generated)",
  "title": "string (required, max 500 chars)",
  "description": "text (optional)",
  "isbn": "string (optional, unique, max 13 chars)",
  "publisher": "string (optional, max 200 chars)",
  "publication_date": "date (optional)",
  "page_count": "integer (optional, positive)",
  "language": "string (optional, max 50 chars)",
  "cover_image_url": "url (optional)",
  "authors": "many-to-many relationship with Author",
  "genres": "many-to-many relationship with Genre",
  "average_rating": "decimal (computed, 0-5)",
  "ratings_count": "integer (computed)",
  "created_at": "datetime (auto-generated)",
  "updated_at": "datetime (auto-updated)"
}
```

---

### Review

```
{
  "id": "integer (auto-generated)",
  "book": "foreign key to Book (required)",
  "user": "foreign key to User (required)",
  "rating": "integer (required, 1-5)",
  "comment": "text (optional)",
  "created_at": "datetime (auto-generated)",
  "updated_at": "datetime (auto-updated)"
}
```

**Constraints:**

- Unique together: (book, user) - One review per user per book

---

### ReadingList

```
{
  "id": "integer (auto-generated)",
  "user": "foreign key to User (required)",
  "book": "foreign key to Book (required)",
  "status": "choice (want_to_read, reading, completed)",
  "added_at": "datetime (auto-generated)"
}
```

**Constraints:**

- Unique together: (user, book)

---

## Error Handling

### HTTP Status Codes

The API uses standard HTTP status codes:

| Code | Description |
|---|---|
| 200 | OK - Request successful |
| 201 | Created - Resource created successfully |
| 204 | No Content - Resource deleted successfully |
| 400 | Bad Request - Invalid input data |
| 401 | Unauthorized - Authentication required |
| 403 | Forbidden - Insufficient permissions |
| 404 | Not Found - Resource not found |
| 500 | Internal Server Error - Server error |

---

### Error Response Format

```
{
  "error": "Error message description",
  "detail": "Detailed error information",
  "field_errors": {
    "field_name": ["Error message for this field"]
```

```
  }
}
```

---

### Common Errors

#### 400 Bad Request

```
{
  "rating": ["Ensure this value is less than or equal to 5."]
}
```

#### 401 Unauthorized

```
{
  "detail": "Authentication credentials were not provided."
}
```

#### 404 Not Found

```
{
  "detail": "Not found."
}
```

---

## Examples

### Example 1: Create a New Book

**Request:**

```
POST /api/books/
Content-Type: application/json

{
  "title": "The Great Gatsby",
  "description": "A novel about the American Dream",
  "isbn": "9780743273565",
  "publisher": "Charles Scribner's Sons",
  "publication_date": "1925-04-10",
  "page_count": 180,
  "language": "English",
  "author_ids": [7],
```

```
  "genre_ids": [6]
}
```

**Response:**

```
{
  "id": 8,
  "title": "The Great Gatsby",
  "authors": [
    {
      "id": 7,
      "name": "F. Scott Fitzgerald"
    }
  ],
  "genres": [
    {
      "id": 6,
      "name": "Classic"
    }
  ],
  "description": "A novel about the American Dream",
  "isbn": "9780743273565",
  "publisher": "Charles Scribner's Sons",
  "publication_date": "1925-04-10",
  "page_count": 180,
  "language": "English",
  "average_rating": null,
  "ratings_count": 0,
  "created_at": "2026-04-19T15:45:00.000000Z"
}
```

---

### Example 2: Search for Books

**Request:**

```
GET /api/search/?q=harry
```

**Response:**

```
{
  "query": "harry",
  "count": 1,
  "results": [
    {
      "id": 1,
      "title": "Harry Potter and the Philosopher's Stone",
```

```
      "authors": [
        {
          "id": 1,
          "name": "J.K. Rowling"
        }
      ],
      "average_rating": "3.00",
      "publication_date": "1997-06-26"
    }
  ]
}
```

---

### Example 3: Add a Review

**Request:**

```
POST /api/reviews/
Authorization: Basic <credentials>
Content-Type: application/json

{
  "book": 1,
  "rating": 5,
  "comment": "Absolutely loved this book! A masterpiece."
}
```

**Response:**

```
{
  "id": 13,
  "book": 1,
  "book_title": "Harry Potter and the Philosopher's Stone",
  "user": {
    "id": 1,
    "username": "currentuser"
  },
  "rating": 5,
  "comment": "Absolutely loved this book! A masterpiece.",
  "created_at": "2026-04-19T15:50:00.000000Z",
  "updated_at": "2026-04-19T15:50:00.000000Z"
}
```

---

### Example 4: Get Recommendations

**Request:**

```
GET /api/books/1/recommendations/
```

**Response:**

```
{
  "book": {
    "id": 1,
    "title": "Harry Potter and the Philosopher's Stone",
    "genres": ["Fantasy", "Young Adult"]
  },
  "recommendations": [
    {
      "id": 5,
      "title": "Similar Fantasy Book",
      "similarity_score": 0.75,
      "average_rating": "4.20"
    }
  ]
}
```

---

## Rate Limiting

Currently, there are no rate limits implemented. For production use, consider implementing rate limiting to prevent abuse.

---

## Pagination

All list endpoints support pagination with the following parameters:

- `page` - Page number (default: 1)
- `page_size` - Items per page (default: 10, max: 100)

**Response includes:**

- `count` - Total number of items
- `next` - URL to next page (null if last page)
- `previous` - URL to previous page (null if first page)
- `results` - Array of items

---

## Filtering and Ordering

### Filtering

Use query parameters to filter results:

```
GET /api/books/?genres=1&language=English
```

### Ordering

Use the `ordering` parameter:

```
GET /api/books/?ordering=-average_rating
```

Prefix with `-` for descending order.

---

## Documentation and Testing

### Interactive Documentation

- **Swagger UI:** http://127.0.0.1:8000/swagger/
- **ReDoc:** http://127.0.0.1:8000/redoc/

### Sample Data

Load sample data for development:

```
python manage.py load_sample_data
```

This creates:

- 7 authors
- 7 genres
- 7 books
- 12 reviews
- 5 sample users

---